topics

1) DLQueue
2) retry mechanisam
3) fan out

==========
KAFKA SHORT NOTES
==========

1) TOPICS AND THEIR USE

- Topic = Logical category where messages are stored.
- Producer publishes to a topic.
- Consumers subscribe to a topic.
- Topic is divided into partitions.

Example:
  order.created
  order.payment.completed

Use:
  - Decouple services
  - Enable async communication
  - Event-driven architecture

2) PUBLISHER SMALL SETUP (SPRING BOOT)
---

application.properties

spring.kafka.bootstrap-servers=localhost:9092

Producer Service

```
@Autowired
private KafkaTemplate<String, Object> kafkaTemplate;

public void publishOrder(Order order) {
    kafkaTemplate.send("order.created", order);
}
```

Flow:
  Order Object

↓
JsonSerializer
↓
Kafka Broker
↓
Stored in Topic Partition

3) CONSUMER SMALL SETUP (SPRING BOOT)
---

application.properties

```
spring.kafka.bootstrap-servers=localhost:9092
```

Consumer Listener

```
@KafkaListener(
    topics = "order.created",
    groupId = "payment-group"
)
public void consumeOrder(Order order) {
    System.out.println(order);
}
```

Flow:
Consumer joins group
↓
Kafka assigns partition
↓
Polls messages
↓
Processes message
↓
Commits offset

4) PARTITIONING AND OFFSET

Partition:
- Topic is divided into partitions.
- Enables parallel processing.
- Ordering guaranteed only inside one partition.

Example:
Topic: order.created

Partition 0:
Offset 0 → Order1
Offset 1 → Order4

Partition 1:
Offset 0 → Order2
Offset 1 → Order5

Offset:
- Sequential number inside partition.
- Tracks consumer progress.
- Stored per (Consumer Group + Partition).

Rule:
Next read = Last committed offset + 1

5) SMALL END-TO-END FLOW

User places order
↓
Order Service
↓
Publish → order.created topic
↓
Kafka stores message in Partition
↓
Payment Service consumes
↓
Payment successful
↓
Publish → order.payment.completed
↓
Delivery Service consumes

6) ZOOKEEPER AND KAFKA SERVER (LOCAL)

Old Setup:
ZooKeeper + Kafka Broker

ZooKeeper:
- Managed metadata
- Leader election
- Broker coordination

Kafka Broker:
- Stores messages
- Handles partitions
- Serves producers & consumers

Modern Setup (Recommended):
Kafka in KRaft mode (No ZooKeeper)

Local:
- Run Kafka via Docker
- Expose port 9092
- Connect via localhost:9092

==========
KEY FORMULA
==========

Partition → Parallelism
Consumer Group → Load balancing
Offset → Progress tracking
Kafka → Event backbone
==========

==========
1) RETRY MECHANISM IN KAFKA
==========

Retry is used when message processing fails temporarily
(DB issue, API failure, network error).

Instead of losing the message, the consumer retries processing.

Flow:

Producer
↓
Kafka Topic
↓
Consumer reads message
↓
Processing fails
↓
Retry message
↓
If retry limit reached → DLQ

Common Retry Approaches

1) Immediate Retry
   - Retry inside consumer code.
   - Not recommended for heavy workloads.

2) Retry Topics (Production Pattern)

order.created
    ↓ fail
order.created.retry.5s
    ↓ fail
order.created.retry.1m
    ↓ fail
order.created.dlq

3) Spring Kafka Retry

```
@RetryableTopic(
  attempts = 3,
  backoff = @Backoff(delay = 5000)
)
```

Used when failures are temporary.

==========
2) OFFSET COMMITTING
==========

Offset = position of a message inside a partition.

Kafka uses offset to track which messages a consumer has processed.

Structure:

Consumer Group + Partition + Offset

Example:

Partition 0

Offset 0 → Order1
Offset 1 → Order2
Offset 2 → Order3

If offset 1 committed → next read starts from offset 2.

Types of Offset Commit

1) Auto Commit

enable.auto.commit=true

Kafka automatically commits offsets periodically.

Risk:
Message may be lost if processing fails after commit.

2) Manual Commit (Recommended)

Process message
↓
Commit offset

Spring Example:

ack.acknowledge()

Ensures message processed before committing offset.

==========
3) DLQ (DEAD LETTER QUEUE)
==========

DLQ stores messages that cannot be processed
even after multiple retries.

Purpose:
- Prevent message loss
- Allow debugging
- Prevent blocking consumer

Flow:

order.created
↓ fail
Retry Topic
↓ fail
Retry Topic
↓ fail
order.created.dlq

DLQ message usually contains:

- original event
- error reason
- retry count
- timestamp

Engineers later inspect DLQ and decide to:

- reprocess
- fix data
- discard message

==========
4) EVENT ID IN KAFKA
==========

Event ID is a unique identifier for each message.

Kafka itself does NOT generate event IDs.
The producer service generates them.

Usually generated using:

UUID.randomUUID()

Example event:

```
{
  "eventId": "a1b2c3",
  "orderId": "ORD123",
  "status": "CREATED"
}
```

Purpose:

- prevent duplicate processing
- track events
- enable idempotency

==========
5) DUPLICATE EVENT ID HANDLING
==========

Kafka can sometimes deliver duplicate messages due to:

- consumer retry

- network failure
- rebalance
- producer retry

To avoid processing duplicates, consumers implement IDEMPOTENCY.

Approach:

Before processing event:

Check if eventId already processed.

Flow:

Consumer receives event
↓
Check eventId in storage
↓
If exists → ignore
If not exists → process event
↓
Store eventId

Where eventId is stored:

1) Database table

processed_events
---
event_id
processed_at

2) Redis (for high throughput)

key = processed_event:{eventId}
TTL = 24 hours

Example logic:

```
if(eventId exists)
   skip processing
else
   process event
   store eventId
```

==========

SUMMARY
==========

Retry → handle temporary failures
Offset → track consumer progress
DLQ → store permanently failed messages
Event ID → uniquely identify event
Duplicate handling → ensure idempotent processing

==========
KAFKA DLQ (Dead Letter Queue) - SHORT NOTE
==========

1) WHEN DOES A MESSAGE GO TO DLQ?

A message is sent to DLQ when the consumer fails to process it after retries.

Common reasons:
- Validation failure (invalid payload)
- Deserialization error (JSON parsing issue)
- Business rule failure (missing required fields)
- External dependency failure (DB/API down)
- Retry attempts exhausted

---

2) HOW DOES IT GO TO DLQ? (SPRING KAFKA)

Spring Kafka uses:
- DefaultErrorHandler
- DeadLetterPublishingRecoverer

Flow:
Consumer → Retry (e.g., 3 times) → Still fails → Sent to DLQ topic

Example config:

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {
   DeadLetterPublishingRecoverer recoverer =
      new DeadLetterPublishingRecoverer(template);

   DefaultErrorHandler handler =
      new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3));
```

```
    return handler;
}
```

---

3) EXAMPLE SCENARIO

Message:

```
{
  "auditId": null
}
```

Consumer logic:

```
if (auditId == null) throw Exception
```

Flow:
main-topic → consumer ❌ → retry → fail → DLQ topic

---

4) HOW TO FETCH MESSAGE FROM DLQ?

Option 1: Kafka Consumer

```
@KafkaListener(topics = "main-topic.DLT")
public void readDlq(String message) {
    System.out.println("DLQ message: " + message);
}
```

Option 2:
- Kafka UI tools
- CLI consumer

---

5) HOW TO REPLAY MESSAGE FROM DLQ?

Replay = produce message again to main topic

```
@KafkaListener(topics = "main-topic.DLT")
public void replay(String message,
                   @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,
                   @Header(KafkaHeaders.RECEIVED_KEY) String key) {

    kafkaTemplate.send("main-topic", key, message);
```

}

---

6) PARTITION HANDLING DURING REPLAY

Case 1: No partition specified
→ Kafka assigns:
  - Based on key (hashing)
  - Or round-robin

Case 2: Preserve original partition (BEST PRACTICE)

```
kafkaTemplate.send(
    new ProducerRecord<>("main-topic", partition, key, message)
);
```

→ Message goes to SAME partition

---

7) OFFSET HANDLING

- Original offset is NOT reused
- Replay creates NEW message

Example:

Before:
Partition 0 → Offset 10 (failed)

After replay:
Partition 0 → Offset 25 (new message)

---

8) HOW TO HANDLE FAILED MESSAGE PROPERLY

Best practices:

1. Idempotency
  - Use unique eventId
  - Avoid duplicate processing

2. Retry limit

- Prevent infinite loops

3. Logging
   - Store error details

4. DLQ metadata
   - original topic, partition, offset

5. Alerting
   - Monitor DLQ size

---

9) FINAL FLOW SUMMARY

Producer → Main Topic → Consumer
→ (Fail) → Retry → Fail
→ DLQ Topic
→ Ops Fix / Auto Replay
→ Produce Again → Main Topic
→ Consumer (Success)

KEY POINT:
Replay ≠ same message retry
Replay = NEW message with same data
